

Instruction Tables

Interleaving the stages of execution is a general idea in digital electronics, and it is applied not only in the main CPU pipeline, but also on the level of separate instructions and [memory](#). Most execution units have their own little pipelines and can take another instruction just one or two cycles after the previous one.

In this context, it makes sense to use two different “[costs](#)” for instructions:

- *Latency*: how many cycles are needed to receive the results of an instruction.
- *Throughput*: how many instructions can be, on average, executed per cycle.

You can get latency and throughput numbers for a specific architecture from special documents called [instruction tables](#). Here are some sample values for my Zen 2 (all specified for 32-bit operands, if there is any difference):

Instruction	Latency	Throughput
<code>jmp</code>	-	2
<code>mov r, r</code>	-	1/4
<code>mov r, m</code>	4	1/2
<code>mov m, r</code>	3	1
<code>add</code>	1	1/3
<code>cmp</code>	1	1/4
<code>popcnt</code>	1	1/4
<code>mul</code>	3	1
<code>div</code>	13-28	13-28

Some comments:

- Because our minds are so used to the cost model where “more” means “worse,” people mostly use *reciprocals* of throughput instead of throughput.
- If a certain instruction is especially frequent, its execution unit could be duplicated to increase its throughput – possibly to even more than one, but not higher than the `decode width`.
- Some instructions have a latency of 0. This means that these instructions are used to control the scheduler and don’t reach the execution stage. They still have non-zero reciprocal throughput because the `CPU front-end` still needs to process them.
- Most instructions are pipelined, and if they have the reciprocal throughput of n , this usually means that their execution unit can take another instruction after n cycles (and if it is below 1, this means that there are multiple execution units, all capable of taking another instruction on the next cycle). One notable exception is `integer division`: it is either very poorly pipelined or not pipelined at all.
- Some instructions have variable latency, depending on not only the size, but also the values of the operands. For memory operations (including fused ones like `add`), the latency is usually specified for the best case (an L1 cache hit).

There are many more important little details, but this mental model will suffice for now.